11/Arrays

An *array* is a list of variables that share a common name. Arrays are useful because they make it possible to work with more variables without creating a new name for each one. This makes the code shorter, easier to read, and more convenient to update.

From Variables to Arrays

When a program needs to keep track of one or two things, it's not necessary to use an array. In fact, adding an array might make the program more complicated than necessary. However, when a program has many elements (for example, a field of stars in a space game or multiple data points in a visualization), arrays make the code easier to write.

Example 11-1: Many Variables

To see what we mean, refer to Example 8-3 on page 123. This code works fine if we're moving around only one shape, but what if we want to have two? We need to make a new `x` variable and update it within `draw()`:

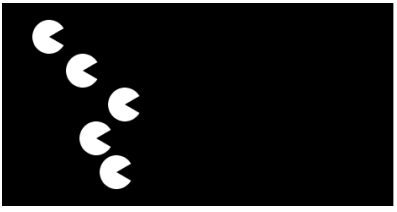
Copyright © 2015. Maker Media, Inc. All rights reserved.

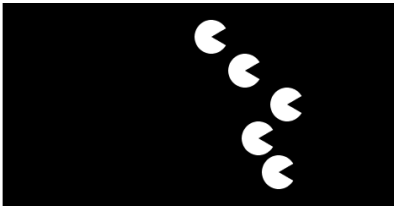
McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

```
var x1 = -20;
var x2 = 20;

function setup() {
  createCanvas(240, 120);
  noStroke();
}

function draw() {
  background(0);
  x1 += 0.5;
  x2 += 0.5;
  arc(x1, 30, 40, 40, 0.52, 5.76);
  arc(x2, 90, 40, 40, 0.52, 5.76);
}
```

Example 11-2: Too Many Variables

The code for the previous example is still manageable, but what if we want to have five circles? We need to add three more variables to the two we already have:





```
var x1 = -10;
var x2 = 10;
var x3 = 35;
var x4 = 18;
var x5 = 30;

function setup() {
  createCanvas(240, 120);
  noStroke();
}

function draw() {
  background(0);
  x1 += 0.5;
  x2 += 0.5;
  x3 += 0.5;
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

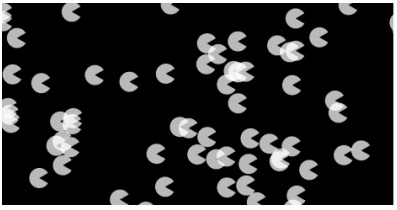
McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

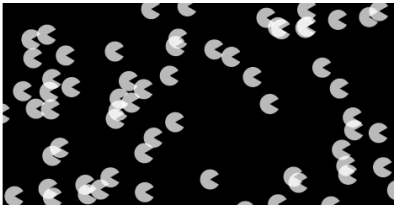
```
  x4 += 0.5;
  x5 += 0.5;
  arc(x1, 20, 20, 20, 0.52, 5.76);
  arc(x2, 40, 20, 20, 0.52, 5.76);
  arc(x3, 60, 20, 20, 0.52, 5.76);
  arc(x4, 80, 20, 20, 0.52, 5.76);
  arc(x5, 100, 20, 20, 0.52, 5.76);
}
```

This code is starting to get out of control!

Example 11-3: Arrays, Not Variables

Imagine what would happen if you wanted to have 3,000 circles. This would mean creating 3,000 individual variables, then updating each one separately. Could you keep track of that many variables? Would you want to? Instead, we use an array:





```
var x = [];

function setup() {
  createCanvas(240, 120);
  noStroke();
  fill(255, 200);
  for (var i = 0; i < 3000; i++) {
    x[i] = random(-1000, 200);
  }
}

function draw() {
  background(0);
  for (var i = 0; i < x.length; i++) {
    x[i] += 0.5;
    var y = i * 0.4;
    arc(x[i], y, 12, 12, 0.52, 5.76);
  }
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

We'll spend the rest of this chapter talking about the details that make this example possible.

Make an Array

Each item in an array is called an *element*, and each has an *index* value to mark its position within the array. Just like coordinates on the canvas, index values for an array start counting from 0. For instance, the first element in the array has the index value 0, the second element in the array has the index value 1, and so on. If there are 20 values in the array, the index value of the last element is 19. Figure 11-1 shows the conceptual structure of an array.

```
var years = [ 1920, 1972, 1980, 1996, 2010 ];
```

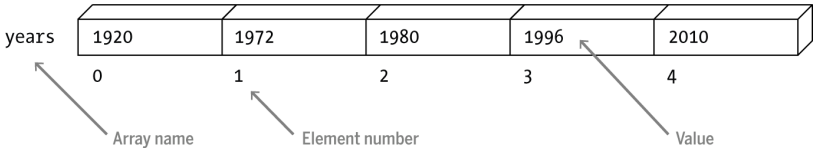


Figure 11-1. *An array is a list of one or more variables that share the same name*

Using arrays is similar to working with single variables; it follows the same patterns. As you know, you can make a single variable called x with this code:

```
var x;
```

To make an array, just set the variable's value to a set of empty brackets:

```
var x = [];
```

Note that the length of the array does not need to be declared in advance; the length is determined by the number of elements you put into it.

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

An array can store all different types of data (`boolean`, `number`, `string`, etc.). You can mix and match different types of data within a single array.

Before we get ahead of ourselves, let's slow down and talk about working with arrays in more detail. There are two steps to working with an array:

1. Declare the array.
2. Assign values to each element.

Each step can happen on its own line, or both the steps can be compressed together. Each of the two following examples shows a different technique to create an array called `x` that stores two numbers, 12 and 2. Pay close attention to what happens before `setup()` and what happens within `setup()`.

Example 11-4: Declare and Assign an Array

First, we'll declare the array outside of `setup()` and then create and assign the values within. The syntax `x[0]` refers to the first element in the array and `x[1]` is the second:

```
var x = [];        // Declare the array

function setup() {
  createCanvas(200, 200);
  x[0] = 12;       // Assign the first value
  x[1] = 2;        // Assign the second value
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

Example 11-5: Assigning to an Array in One Go

You can also assign values to the array when it's created, if it's all part of a single statement:

```
var x = [12, 2]; // Declare and assign

function setup() {
  createCanvas(200, 200);
}
```

Avoid creating arrays within `draw()`, because creating a new array on every frame will slow down your frame rate.

Example 11-6: Revisiting the First Example

As a complete example of how to use arrays, we've recoded Example 11-1 on page 165 here. Although we don't yet see the full benefits revealed in Example 11-3 on page 167, we do see some important details of how arrays work:

```
var x = [-20, 20];

function setup() {
  createCanvas(240, 120);
  noStroke();
}

function draw() {
  background(0);
  x[0] += 0.5;  // Increase the first element
  x[1] += 0.5;  // Increase the second element
  arc(x[0], 30, 40, 40, 0.52, 5.76);
  arc(x[1], 90, 40, 40, 0.52, 5.76);
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

Repetition and Arrays

The `for` loop, introduced in "Repetition" on page 46, makes it easier to work with large arrays while keeping the code concise. The idea is to write a loop to move through each element of the array one by one. To do this, you need to know the length of the array. The `length` property associated with each array stores the number of elements. We use the name of the array with the dot operator (a period) to access this value. For instance:

```
var x = [12, 20];          // Declare and assign the array
print(x.length);           // Prints 2 to the console

var y = ["cat", 10, false, 50]; // Declare and assign the array
print(y.length);           // Prints 4 to the console

var z = [];                // Declare an empty array
print(z.length);           // Prints 0 to the console
z[0] = 20;                 // Assign an element to the array
print(z.length);           // Prints 1 to the console
z[1] = 4;                  // Assign an element to the array
print(z.length);           // Prints 2 to the console
```

Example 11-7: Filling an Array in a for Loop

A `for` loop can be used to fill an array with values, or to read the values back out. In this example, the array is first filled with random numbers inside `setup()`, and then these numbers are used to set the stroke value inside `draw()`. Each time the program is run, a new set of random numbers is put into the array:

```
var gray = [];

function setup() {
  createCanvas(240, 120);
  for (var i = 0; i < width; i++) {
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

```
    gray[i] = random(0, 255);
  }
}

function draw() {
  background(204);
  for (var i = 0; i < gray.length; i++) {
    stroke(gray[i]);
    line(i, 0, i, height);
  }
}
```

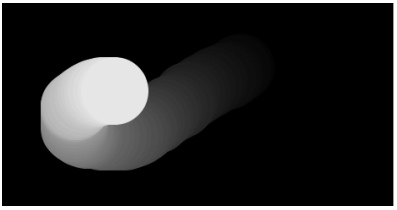
In `setup()`, we insert as many elements as the width of the canvas. This is an arbitrary number; we've chosen it so that drawing a vertical line for each element fills the width of the canvas. You could try changing `width` to read any number. Once the elements are assigned to the array, we are able to iterate through them in `draw()` using the `length` property. We can't use this to iterate through the array in `setup()` because before anything is put into it, the length of array `gray` is 0.

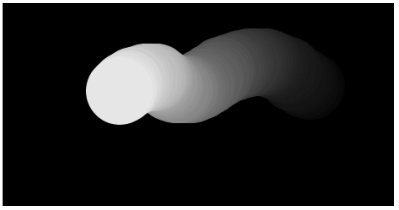
Example 11-8: Track Mouse Movements

In this example, there are two arrays to store the position of the mouse—one for the *x* coordinate and one for the *y* coordinate. These arrays store the location of the mouse for the previous 60 frames. With each new frame, the oldest *x* and *y* coordinate values are removed and replaced with the current `mouseX` and `mouseY` values. The new values are added to the first position of the array, but before this happens, each value in the array is moved one position to the right (from back to front) to make room for the new numbers. (See Figure 11-2 for a diagram illustrating this process.) This example visualizes this action. Also, at each frame, all 60 coordinates are used to draw a series of ellipses to the screen:

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.





```
var num = 60;
var x = [];
var y = [];

function setup() {
  createCanvas(240, 120);
  noStroke();

  for (var i = 0; i < num; i++) {
    x[i] = 0;
    y[i] = 0;
  }
}

function draw() {
  background(0);
  // Copy array values from back to front
  for (var i = num-1; i > 0; i--) {
    x[i] = x[i-1];
    y[i] = y[i-1];
  }
  x[0] = mouseX; // Set the first element
  y[0] = mouseY; // Set the first element
  for (var i = 0; i < num; i++) {
    fill(i * 4);
    ellipse(x[i], y[i], 40, 40);
  }
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

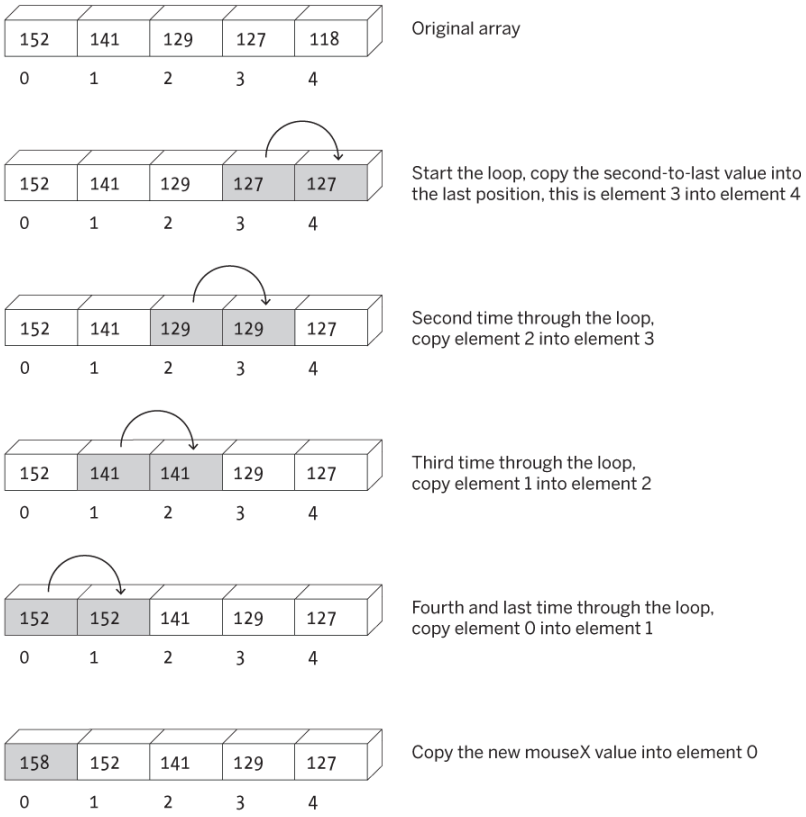


Figure 11-2. *Shifting the values in an array one place to the right*

Arrays of Objects

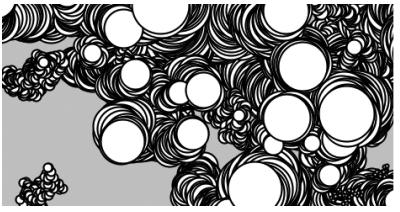
The two short examples in this section bring together every major programming concept in this book: variables, iteration, conditionals, functions, objects, and arrays. Making an array of objects is nearly the same as making the arrays we introduced on the previous pages, but there's one additional consideration: because each array element is an object, it must first be created with the keyword `new` (like any other object) before it is assigned to the array. With a custom-defined object such as `JitterBug` (see Chapter 10), this means using `new` to set up each element before it's assigned to the array.

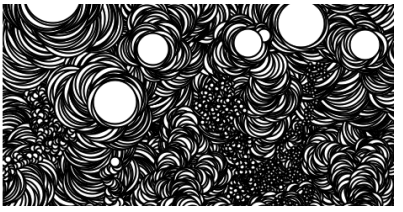
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

Example 11-9: Managing Many Objects

This example creates an array of 33 `JitterBug` objects and then updates and displays each one inside `draw()`. For this example to work, you need to add the `JitterBug` constructor function to the code:





```
var bugs = [];

function setup() {
  createCanvas(240, 120);
  background(204);
  for (var i = 0; i < 33; i++) {
    var x = random(width);
    var y = random(height);
    var r = i + 2;
    bugs[i] = new JitterBug(x, y, r);
  }
}

function draw() {
  for (var i = 0; i < bugs.length; i++) {
    bugs[i].move();
    bugs[i].display();
  }
}

// Copy JitterBug constructor function here
```

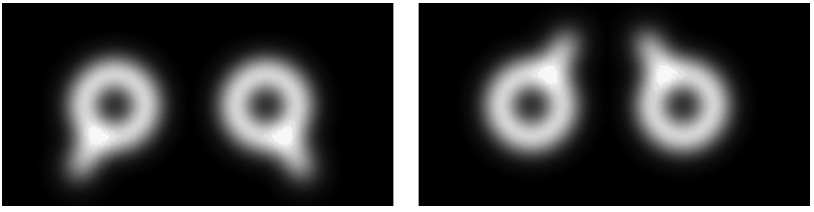
The final array example loads a sequence of images and stores each as an element within an array.

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

Example 11-10: Sequences of Images

To run this example, get the images from the *media.zip* file as described in Chapter 7. The images are named sequentially (*frame-0000.png*, *frame-0001.png*, etc.), which makes it possible to create the name of each file within a `for` loop, as seen in the seventh line of the program:



```
var numFrames = 12; // The number of frames
var images = []; // Make the array
var currentFrame = 0;

function preload() {
  for (var i = 0; i < numFrames; i++) {
    var imageName = "frame-" + nf(i, 4) + ".png";
    images[i] = loadImage(imageName); // Load each image
  }
}

function setup() {
  createCanvas(240, 120);
  frameRate(24);
}

function draw() {
  image(images[currentFrame], 0, 0);
  currentFrame++; // Next frame
  if (currentFrame == images.length) {
    currentFrame = 0;  // Return to first frame
  }
}
```

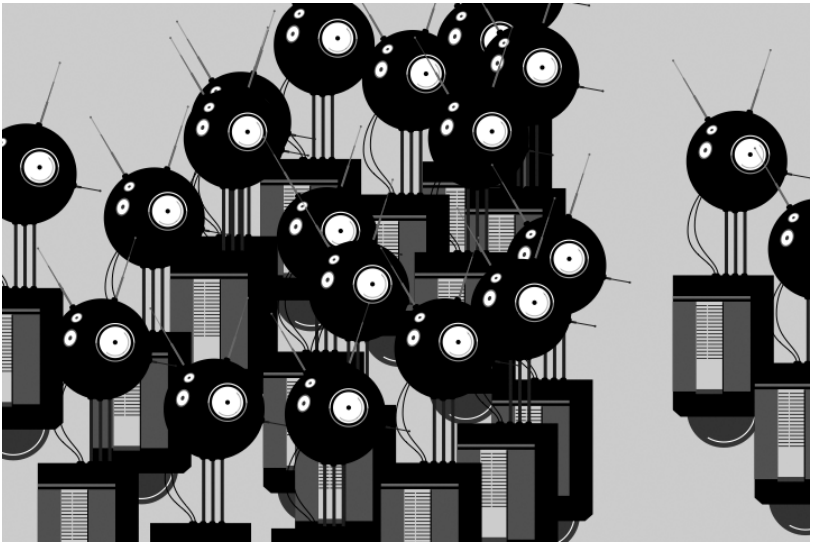
The `nf()` function formats numbers so that `nf(1, 4)` returns the string "0001" and `nf(11, 4)` returns "0011". These values are concatenated with the beginning of the filename (*frame-*) and the end (*.png*) to create the complete filename stored in a variable. The files are loaded into the array on the following line. The

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

images are displayed to the screen one at a time in `draw()`. When the last image in the array is displayed, the program returns to the beginning of the array and shows the images again in sequence.

Robot 9: Arrays



Arrays make it easier for a program to work with many elements. In this example, an array of `Robot` objects is declared at the top. The array is then allocated inside `setup()`, and each `Robot` object is created inside the `for` loop. In `draw()`, another `for` loop is used to update and display each element of the `bots` array.

The `for` loop and an array make a powerful combination. Notice the subtle differences between the code for this example and Robot 8 (see "Robot 8: Objects" on page 162) in contrast to the extreme changes in the visual result. Once an array is created and a `for` loop is put in place, it's as easy to work with 3 elements as it is 3,000.

```
var robotImage;
var bots = [];  // Declare array to hold Robot objects

function preload() {
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

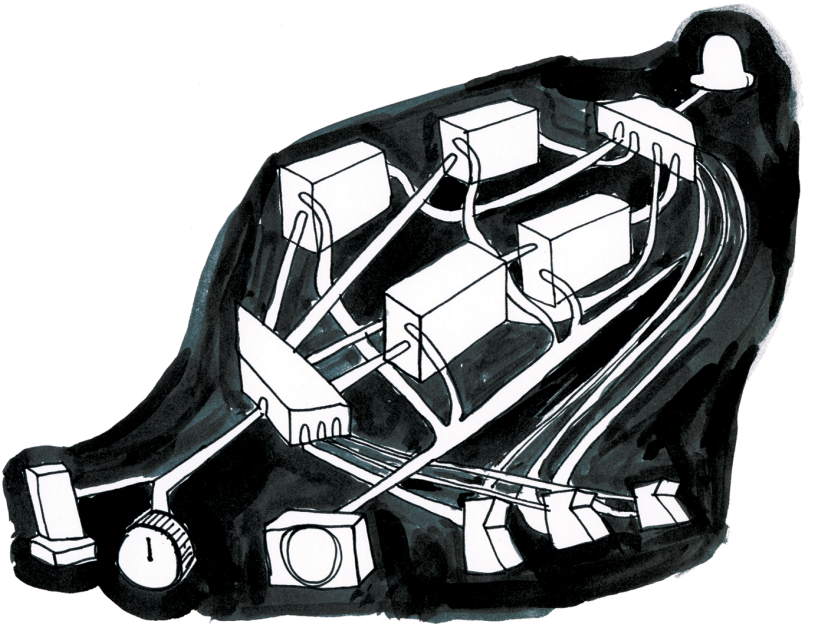
```
  robotImage = loadImage("robot1.svg");
}

function setup() {
  createCanvas(720, 480);

  var numRobots = 20;

  // Create each object
  for (var i = 0; i < numRobots; i++) {
    // Create a random x coordinate
    var x = random(-40, width-40);
    // Assign the y coordinate based on the order
    var y = map(i, 0, numRobots, -100, height-200);
    bots[i] = new Robot(robotImage, x, y);
  }
}

function draw() {
  background(204);
  // Update and display each bot in the array
  for (var i = 0; i < bots.length; i++) {
    bots[i].update();
    bots[i].display();
  }
}

function Robot(img, tempX, tempY) {
  // Set initial values for properties
  this.xpos = tempX;
  this.ypos = tempY;
  this.angle = random(0, TWO_PI);
  this.botImage = img;
  this.yoffset = 0.0;

 // Update the properties
  this.update = function() {
    this.angle += 0.05;
    this.yoffset = sin(this.angle) * 20;
  }

  // Draw the robot to the screen
  this.display = function() {
    image(this.botImage, this.xpos, this.ypos + this.yoffset);
  }
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.



Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-12 14:24:38.